K-Means & Isolation Forest

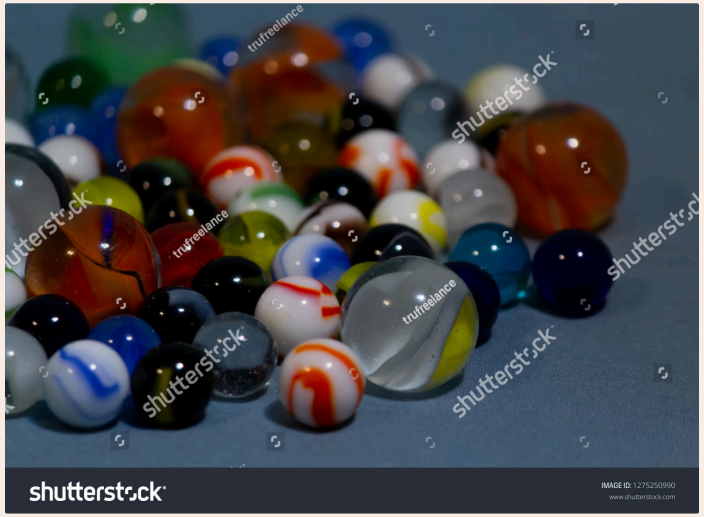
A hands-on study guide for beginners — with real examples, Python code, and exam traps to avoid. Think of this as your learning notebook, not a textbook. Let's cluster some data! 🎯

BEGINNER FRIENDLY | PYTHON + SKLEARN | STEP-BY-STEP



Chapter 1

K-Means: The Core Intuition



Imagine you tip a bag of **mixed marbles** onto a table. You naturally start sorting them into piles by colour. That's K-Means — it groups data points so that **nearby points share the same cluster**.

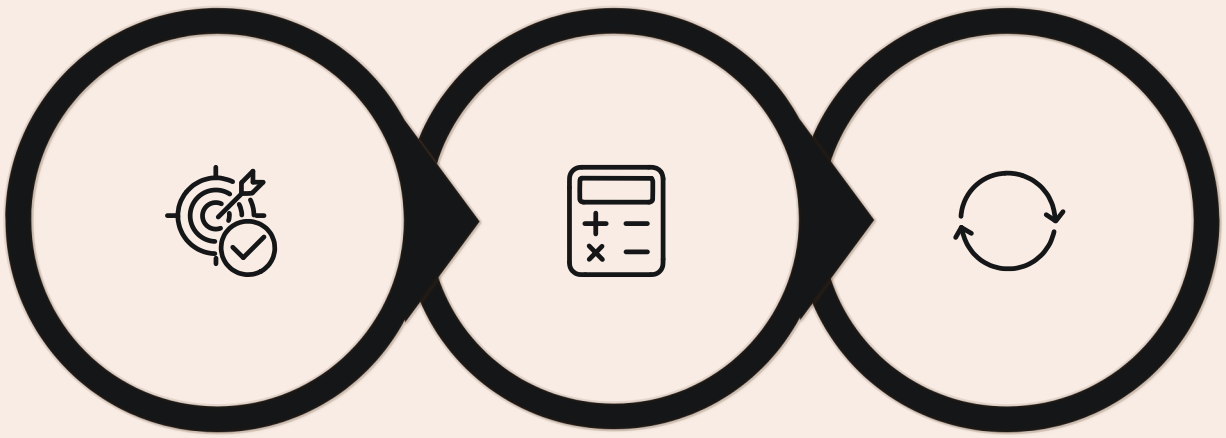
> K-Means doesn't know what the groups *mean* — it only knows distances. You decide how many clusters (K) to create.

- Pick K starting points (centroids)
- Assign every point to its nearest centroid
- Move each centroid to the average of its cluster
- Repeat until nothing changes

Made with GAMMA

Manual Example: Step by Step

Let's cluster 4 points by hand with **K=2**. Starting centroids: **C1=(1,1)** and **C2=(5,5)**



Assign **Recompute** **Repeat**

Points: A=(1,2), B=(2,1), C=(5,4), D=(4,5). Calculate distance from each point to C1 and C2, assign to the closer one, then average the coordinates to get the new centroid.

Point	Dist to C1(1,1)	Dist to C2(5,5)	Assigned to
A=(1,2)	1.00	5.00	C1 ✅
B=(2,1)	1.00	5.00	C1 ✅
C=(5,4)	5.00	1.00	C2 ✅
D=(4,5)	5.00	1.00	C2 ✅

New C1 = mean of A,B = **(1.5, 1.5)** — New C2 = mean of C,D = **(4.5, 4.5)**. Converged in 1 step!

Made with GAMMA

Inertia & The Elbow Method

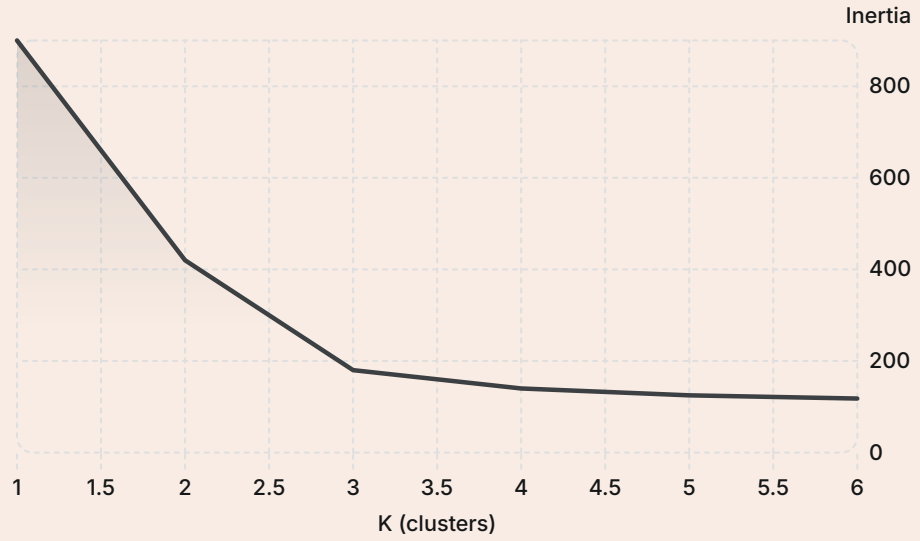
What is Inertia?

Inertia = the **total squared distance** from each point to its centroid. Lower inertia = tighter clusters. But more clusters always means lower inertia — so how do you pick the right K?

> Inertia formula: **Σ (distance from point to its centroid)²** summed across all clusters.

The **Elbow Method**: plot inertia vs. K. The "elbow" — where the curve bends sharply — is your sweet spot. Adding more clusters beyond that point gives diminishing returns.

> ⚠️ Exam trap: The elbow isn't always obvious. Sometimes you'll need to use judgement or silhouette scores to confirm your choice.



Elbow is at **K=3** — the biggest bend. Beyond K=3, inertia barely drops.

Made with GAMMA

Real-World Example: Boutique Customer Segmentation

You run a boutique clothing shop and want to group customers by **annual spend** and **visit frequency**. K-Means finds natural customer segments so you can personalise your marketing.

🛍️ Cluster 1 — Loyalists

High visits, high spend. Your VIPs — treat them to exclusive previews and loyalty perks.

💸 Cluster 2 — Big Spenders

Low visits, high spend. They come rarely but buy big. Target with seasonal promotions.

👀 Cluster 3 — Browsers

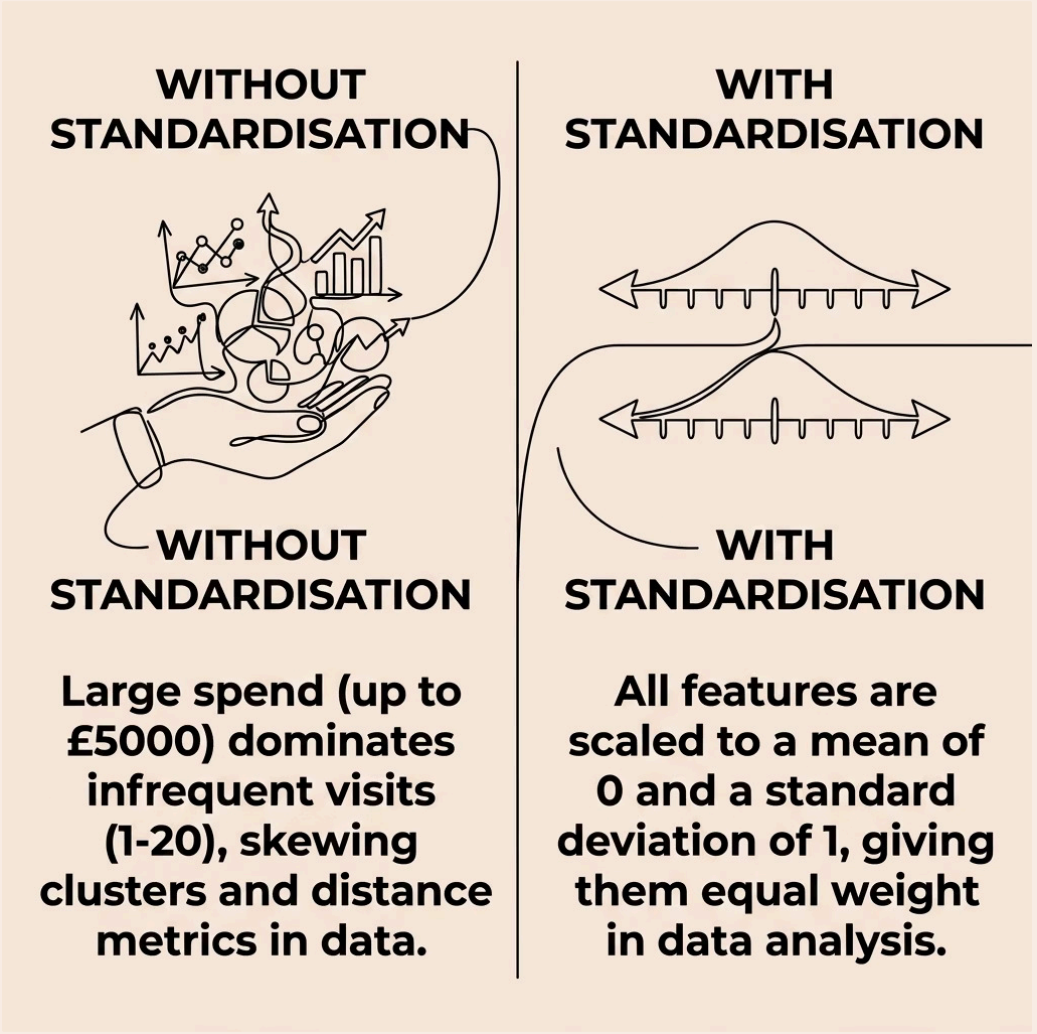
High visits, low spend. They love the store but hesitate. Win them with small discounts or bundles.

ⓘ Before running K-Means, always **standardise your features** — otherwise "spend in £" will dominate "visit count" just because of its larger scale.



Made with GAMMA

Why Standardisation Matters



The Golden Rule

K-Means uses **Euclidean distance**. A feature measured in thousands will dominate one measured in units — even if both are equally important.

Fix it with **StandardScaler**:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

⚠️ ⚠️ Common mistake: Fitting the scaler on test data separately. Always `fit` on training data only, then `transform` both sets.

Made with GAMMA

K-Means in Python

Here's a complete, commented sklearn example using our boutique dataset. Copy it, run it, tweak K — and watch the clusters shift.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

# Sample data: [annual_spend, visit_frequency]
X = np.array([[500,2],[4800,15],[200,12],
              [3200,8],[150,18],[4500,14]])

# Step 1: Standardise
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Step 2: Fit K-Means
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)

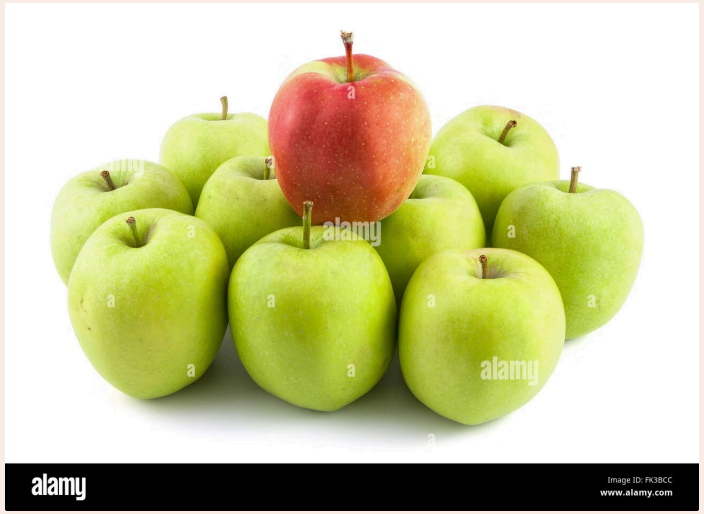
# Step 3: Inspect results
print("Labels:", kmeans.labels_)
print("Inertia:", kmeans.inertia_)
print("Centroids:", kmeans.cluster_centers_)
```

Set `random_state=42` to get reproducible results — K-Means starts with random centroids, so results can vary between runs otherwise.

Made with GAMMA

Chapter 2

Isolation Forest: Spotting the Odd One Out



Anomalies are **rare and different** — easy to isolate from the crowd. Isolation Forest exploits this: it builds random decision trees and measures how quickly a point gets isolated.

- **Normal points** need many splits to isolate — they're buried deep in the data
- **Anomalies** get isolated in just a few splits — they're alone in sparse regions

> Key parameter: `contamination` — the expected fraction of outliers in your dataset. Default is `0.1` (10%).

Score of **-1 = anomaly**, score of **+1 = normal**. No labels needed — this is **unsupervised** learning.

Made with GAMMA

Isolation Forest in Python + Common Traps

Python Example

```
from sklearn.ensemble import IsolationForest
import numpy as np

# Boutique: flag suspicious transactions
spend = np.array([[500],[480],[520],
                  [490],[12000],[510]])

iso = IsolationForest(
    contamination=0.15,
    random_state=42)
iso.fit(spend)

preds = iso.predict(spend)
# -1 = anomaly, 1 = normal
print(preds)
```

Output: `[1,1,1,1,-1,1]` — £12,000 flagged correctly as an anomaly! 🎉

⚠️ Exam Traps to Avoid

K-Means needs K upfront

You must choose K before fitting. Use the elbow method — never guess.

Centroids ≠ data points

After recalculation, centroids are means — they may not match any actual observation.

Isolation Forest ≠ classification

It's unsupervised. It detects anomalies without ever seeing labelled examples.

Standardise before K-Means

Skip this step and your clusters will be driven by scale, not meaning.

Made with GAMMA



Practice Exercises & Key Takeaways

01

Exercise: Manual K-Means

Points: (2,3),(3,2),(8,8),(9,7). K=2, centroids C1=(2,2), C2=(9,9). Assign, recalculate, repeat. What are the final centroids?

02

Exercise: Elbow Method

You get inertias: K1=800, K2=400, K3=200, K4=190, K5=185. Which K would you choose and why?

03

Exercise: Isolation Forest

Transactions: £50, £48, £52, £8000, £55. Set contamination=0.2 and predict. Which transaction is flagged?

✅ Answers: 1) C1=(2.5,2.5), C2=(8.5,7.5) — 2) K=3, the biggest inertia drop — 3) £8,000 is flagged as -1

Made with GAMMA